Comparison Between Data Update Methods

ELAHEH TOULABI NEJAD | 9631243

Updating Old Data

Updating Data

When working with a database, there may be times when we need to change data that's already been inserted into it. For example, we may need to correct a misspelled entry or perhaps you have new information to add to an incomplete record. Structured Query Language — more commonly known as SQL — provides the ability which allows users to change existing data in a table.

Challenges

Updating data can be done in various ways such as row by row, one big batch, in several smaller batches and etc. which can have different performances (based on run time) that we are going to discuss in this presentation.

Setup SQL Server *Test Environment*

First, we'll setup a SQL Server database table in order to run UPDATE operations.
We create two tables named "LargeBooks" and "SmallBooks" with columns below having in order 10M and 100K rows.

- Id int
- Name varchar(500)
- Author varchar(255)
- Price int
- Edition int
- book_desc text

UPDATE Scenarios

UPDATE Statement

Updating Tables Using Ordinary UPDATE/WHERE Commands

Delete / Insert

Deleting Old Row(s) And Inserting New Row(s)

Iterative

Updating Tables Using a Row By Row Method With a Loop

Test Conditions

Size Of Table

- Small Tablee
- Large Tables

Index

- Updating Tables With Indexes on ‘Id’ and ‘Edition’ Columns
- Updating Tables Without Indexes

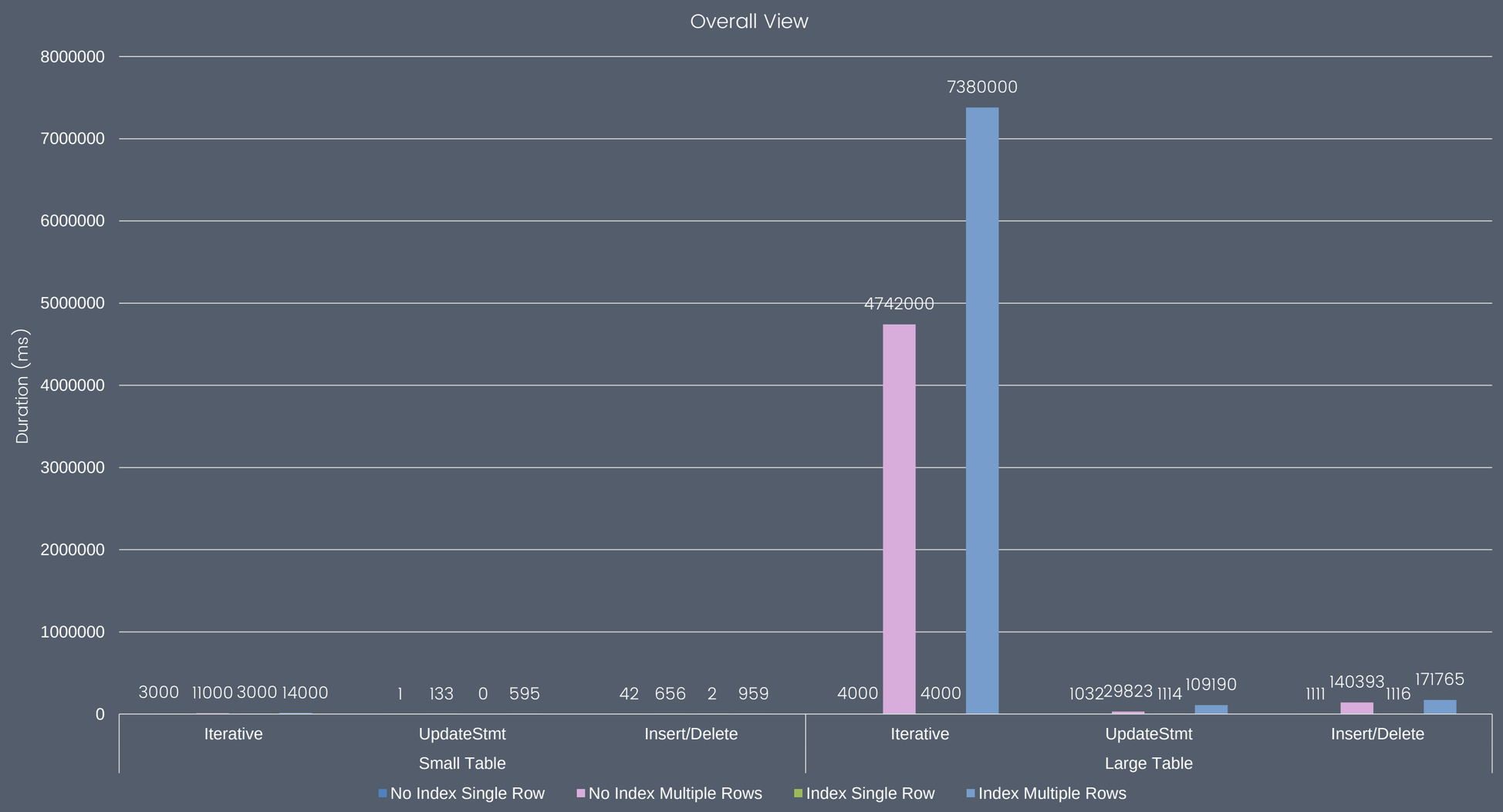
Number Of Rows Being Updated

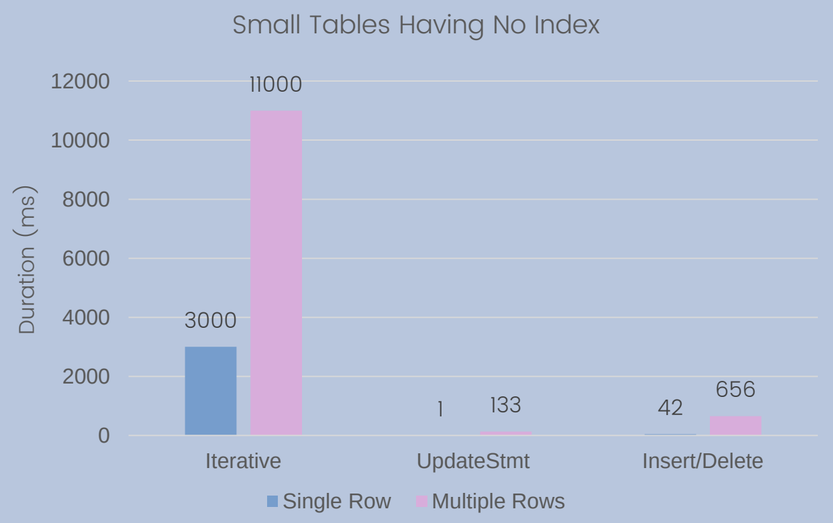
- Updating A Single Row
- Updating Multiple Rows

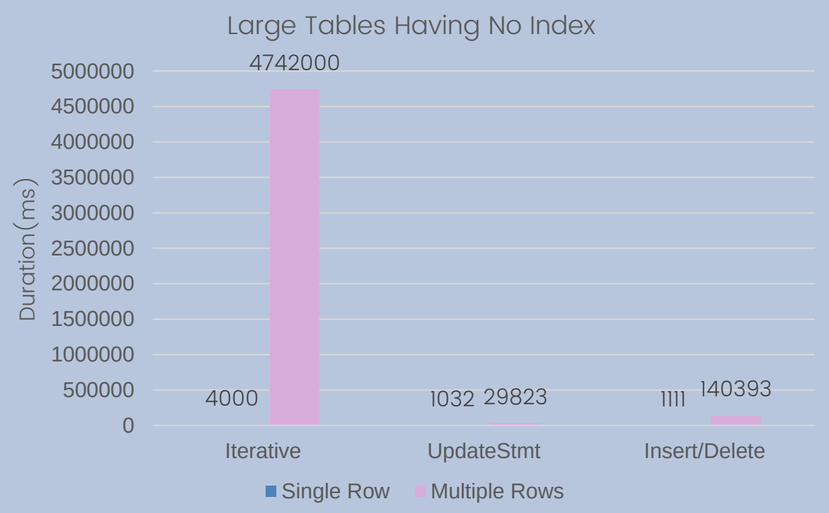
Results Summary

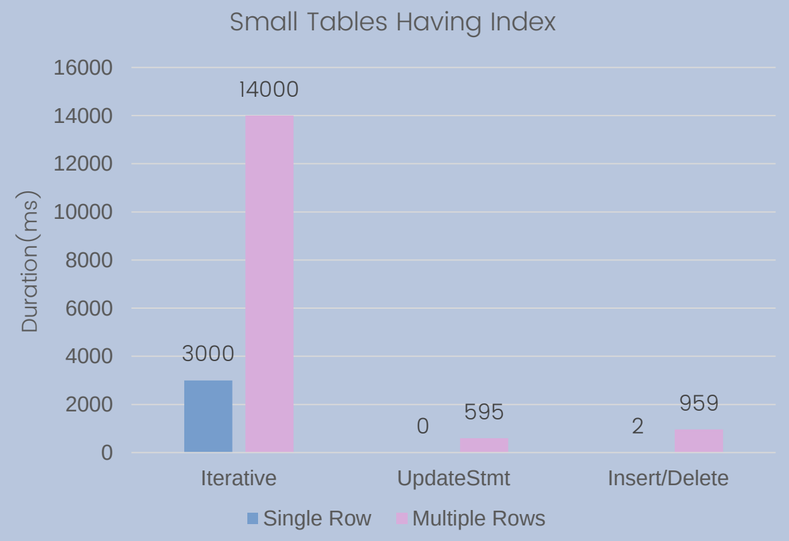
		Small Table			Large Table		
		Iterative	Update Statement	Insert/Delete	Iterative	Update Statement	Insert/Delete
No Index	Single Row	4000	1032	1111	4000	1032	1111
	Multiple Rows	4742000	29823	140393	4742000	29823	140393
Index	Single Row	4000	1114	1116	4000	1114	1116
	Multiple Rows	7380000	109190	171765	>> 7380000	109190	171765

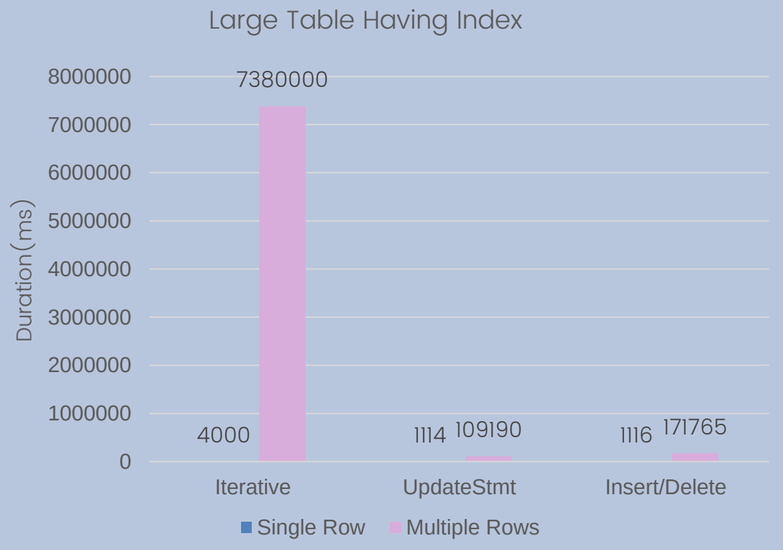
* ms

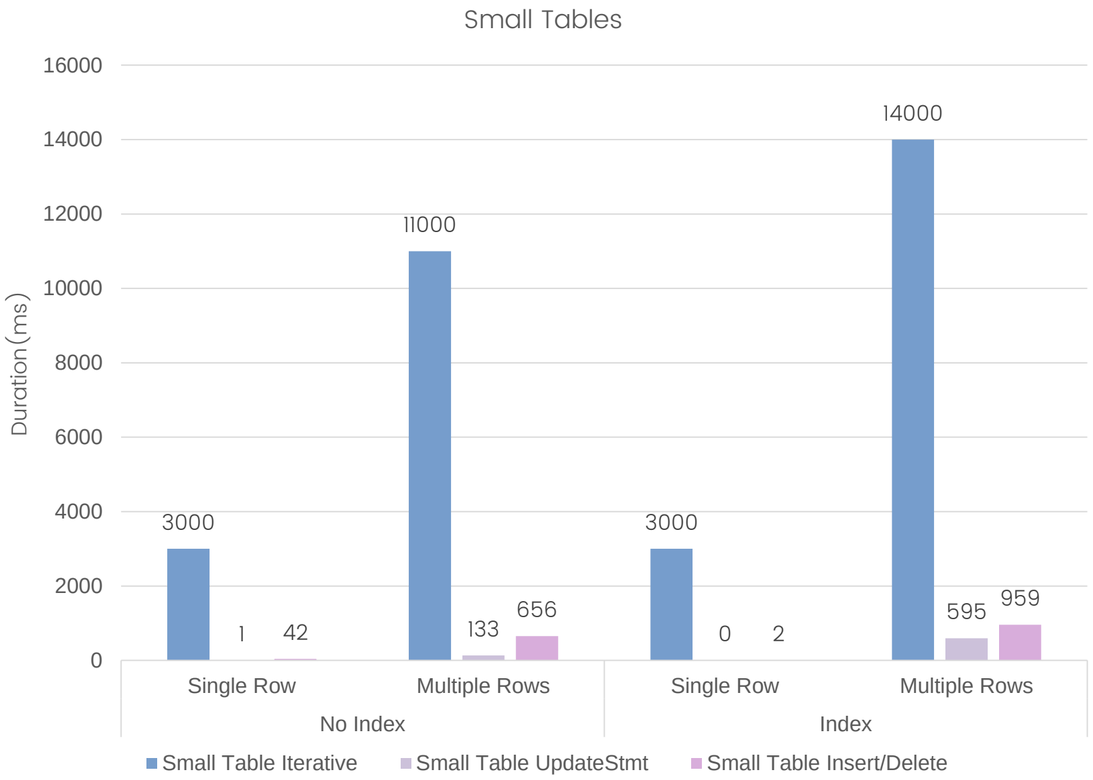

Overall View
Duration (ms)
8000000
7000000
6000000
5000000
4000000
3000000
2000000
1000000
0
3000
11000
3000
14000
1
133
0
595
42
656
2
959
4000
4742000
4000
7380000
1032
29823
1114
109190
1111
140393
1116
171765
Iterative
UpdateStmt
Insert/Delete
Small Table
Iterative
UpdateStmt
Insert/Delete
Large Table
No Index Single Row
No Index Multiple Rows
Index Single Row
Index Multiple Rows

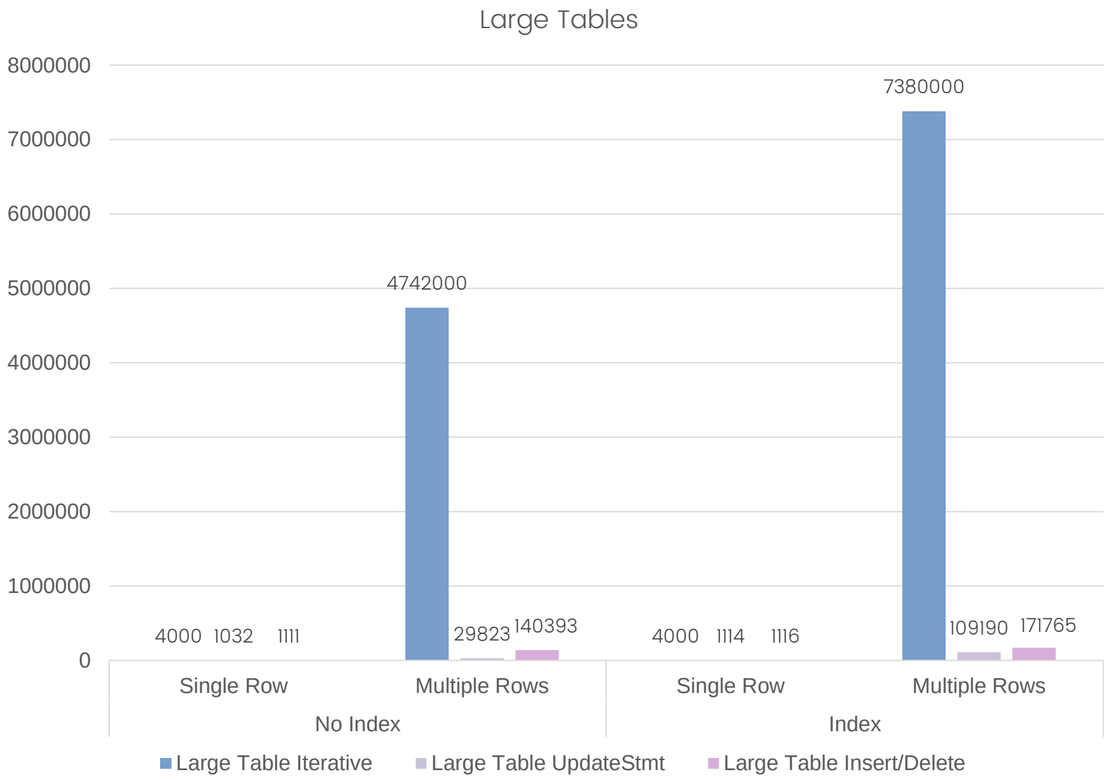

Small Tables Having No Index
Duration (ms)
12000
10000
8000
6000
4000
2000
0
3000
11000
1
133
42
656
Iterative
UpdateStmt
Insert/Delete
Single Row
Multiple Rows

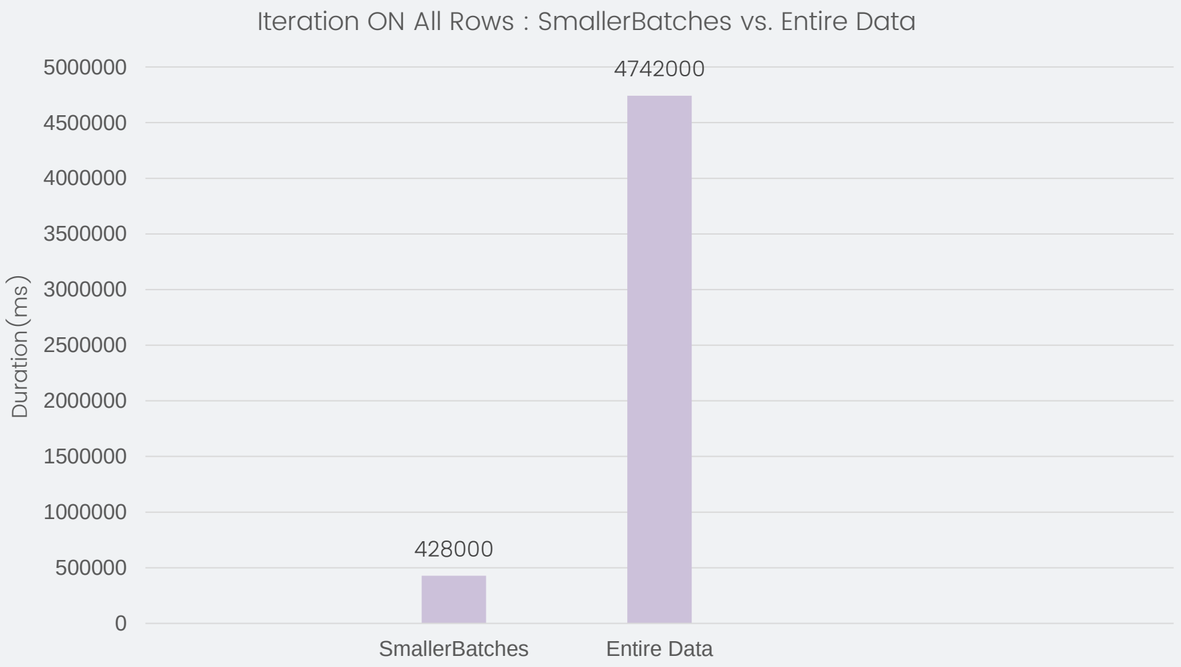

Large Tables Having No Index
Duration(ms)
5000000
4500000
4000000
3500000
3000000
2500000
2000000
1500000
1000000
500000
0
4000
4742000
1032
29823
1111
140393
Iterative
UpdateStmt
Insert/Delete
Single Row
Multiple Rows


Small Tables Having Index
Duration(ms)
16000
14000
12000
10000
8000
6000
4000
2000
0
3000
14000
0
595
2
959
Iterative
UpdateStmt
Insert/Delete
Single Row
Multiple Rows


Large Table Having Index
Duration(ms)
8000000
7000000
6000000
5000000
4000000
3000000
2000000
1000000
0
4000
7380000
1114
109190
1116
171765
Iterative
UpdateStmt
Insert/Delete
Single Row
Multiple Rows


Small Tables
Duration (ms)
16000
14000
12000
10000
8000
6000
4000
2000
0
3000
1
42
11000
133
656
3000
0
2
14000
595
959
Single Row
Multiple Rows
Single Row
Multiple Rows
No Index
Index
Small Table Iterative
Small Table UpdateStmt
Small Table Insert/Delete


Large Tables
8000000
7000000
6000000
5000000
4000000
3000000
2000000
1000000
0
4000
1032
1111
4742000
29823
140393
4000
1114
1116
7380000
109190
171765
Single Row
Multiple Rows
Single Row
Multiple Rows
No Index
Index
Large Table Iterative
Large Table UpdateStmt
Large Table Insert/Delete

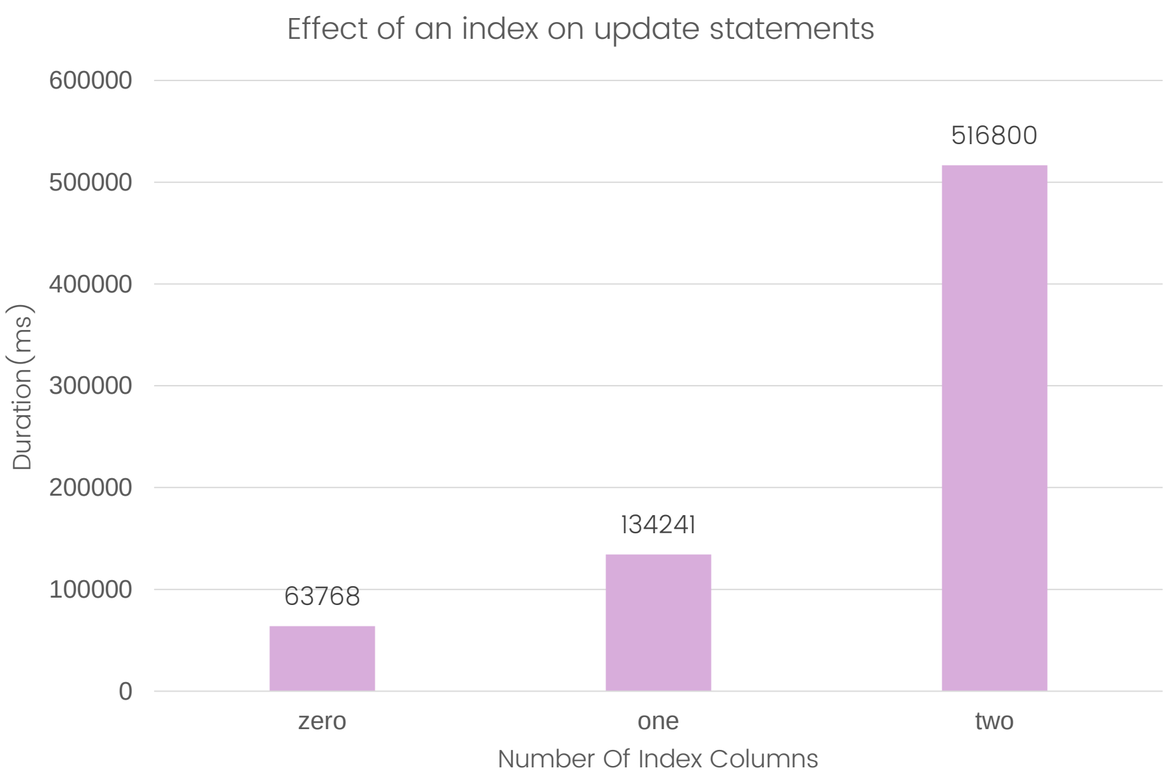


How It Works

This update method devides data into smaller batches and updates one of them in each iteration.

Effect On Performance

Execution time decreases.



How It Works

An update statement must relocate the changed index entries to maintain the index order. For that, the database must remove the old entry and add the new one at the new location.

Effect of an index on update statements

The update performance depends on the number of indexes on the table. the response time grows with each additional index. The response time of the update statement that affects only one index does not increase so much because it leaves most indexes unchanged. (Simple Update Query!)

Iterative Method

Updating row by row is very slow, but less resource intensive and the consistency is at a statement level. If you want all updates to be consistent for the table, you would need to use a transaction, because if one update fails, the rest will succeed. Also, in a transaction when updating row by row with a large volume, it might be possible that deadlocks occur because row iteration in the transaction takes more time.

Update Statement

It's a faster update than a row by row operation, but this is best used when updating limited rows. This is an expensive operation in terms of query cost, because it takes more resources for the single update operation. It also takes time for the update to be logged in the transaction log. Also, long running updates can cause blocking issues for other processes.

Delete And Insert

It's slower than using a single UPDATE statement because it uses a temorary table. It also may cause inconsistency in datas or "holes" in Ids.

- For optimizing update operations you should try to minimize the transaction size.
- Make sure you have the correct indexes in place to optimize the update. If you have indexes that are not being used for the table, removing them could help optimize the update.
- Update method is much more critical when large amount of data are being updated.